

**Computer
Vision**
AI-4002

Lab 05

Image Segmentation

National University of Computer & Emerging Sciences –
NUCES – Karachi



National University of Computer & Emerging Sciences - NUCES - Karachi

Department of Artificial Intelligence

Course Code: AI-4002	Computer Vision Lab
-----------------------------	----------------------------

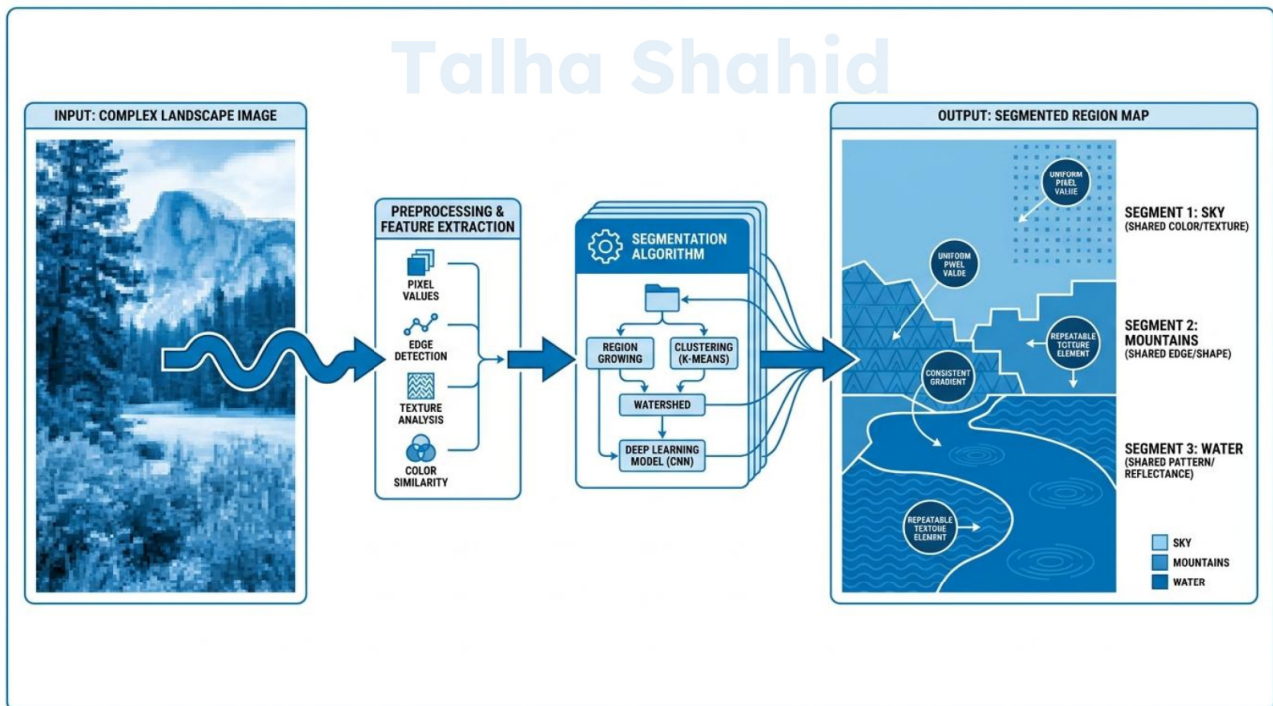
1. Objectives	2
2. Image Segmentation	2
2.1 Mathematics Behind Image Segmentation	4
2.2 Real-Life Examples	5
2.3 Image Segmentation Techniques	7
2.3.1 Thresholding	7
2.3.1.1 Global Thresholding	7
2.3.1.2 Adaptive Thresholding	7
2.3.1.3 Otsu's Thresholding	8
2.3.1.4 Color-Based Thresholding	8
2.3.1.5 Hysteresis Thresholding (Canny Edge Detector):	9
2.3.2 Edge Detection	9
2.3.3 Region Growing	9
2.3.4 Watershed Segmentation	11
2.3.5 Clustering-Based Segmentation	13
2.3.6 Deep Learning-Based Segmentation	14
2.3.7 Active Contour Models (Snakes)	15

1. Objectives

1. Introduction to Image Segmentation
2. Techniques/Methodologies of Image Segmentations
3. Region-based segmentation
4. Edge detection segmentation
5. Clustering-based segmentation

2. Image Segmentation

Image Segmentation is the process of partitioning an image into distinct, non-overlapping regions or segments, each of which corresponds to a meaningful object or part of the scene. The goal is to separate an image into areas that share similar visual characteristics, such as color, texture, or intensity.





National University of Computer & Emerging Sciences - NUCES - Karachi

Department of Artificial Intelligence



Original Image

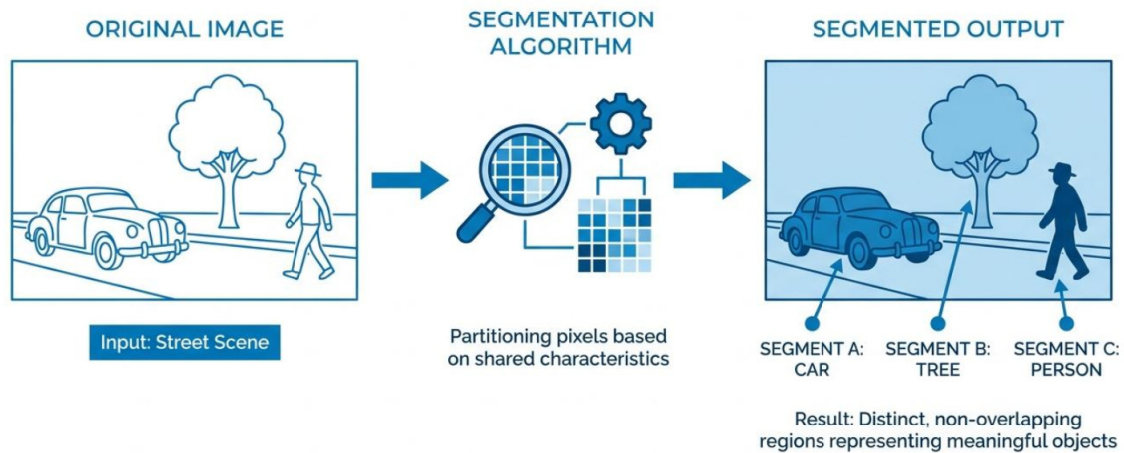


Hair Region Face Region Clothing Region

Segmented Image
(Partitioned Regions)

Process of dividing an image into meaningful, separate parts.

IMAGE SEGMENTATION PROCESS





2.1 Mathematics Behind Image Segmentation

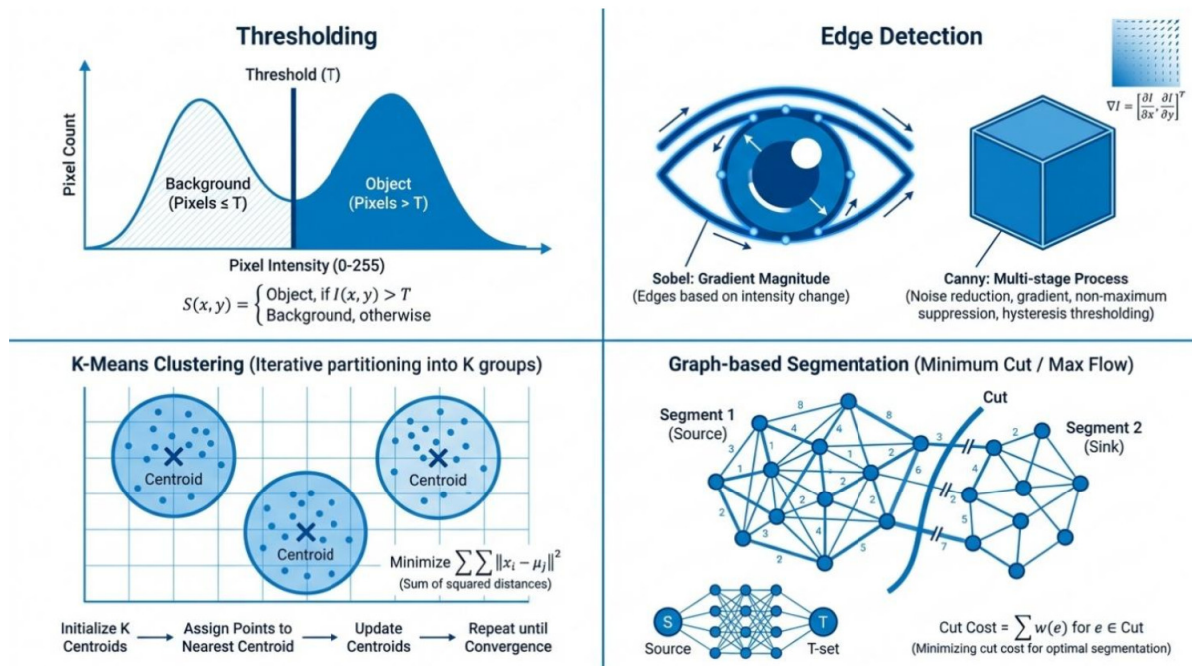
The mathematics behind image segmentation involves various techniques and algorithms, including:

Thresholding: Image segmentation can begin with simple thresholding, where a pixel's intensity value is compared to a predefined threshold. If the intensity is above the threshold, it belongs to one region; otherwise, it belongs to another.

Edge Detection: Edge detection algorithms identify rapid changes in intensity, which often correspond to object boundaries. Common edge detection methods include the Sobel operator, Canny edge detector, and Laplacian of Gaussian (LoG).

Clustering: Clustering techniques, like K-Means clustering or Mean-Shift clustering, group similar pixels together based on feature similarity. These methods consider multiple attributes, such as color values, to form clusters.

Graph Theory: In graph-based segmentation, pixels are represented as nodes in a graph, and edges between nodes represent the similarity between pixel values. Graph algorithms like minimum spanning trees or graph cuts can be used to partition the image into segments.



2.2 Real-Life Examples

- Medical Imaging:** In medical image analysis, segmentation is used to identify and isolate specific anatomical structures or tumors in X-rays, CT scans, or MRIs.
- Object Detection:** In autonomous vehicles, image segmentation is employed to detect and locate objects like pedestrians, other vehicles, and road signs.
- Satellite Imagery:** Segmentation of satellite images can help classify land cover types, monitor deforestation, and assess urban growth.
- Biomedical Imaging:** In microscopy images, segmentation is used to count cells, identify cell nuclei, and diagnose diseases.
- Augmented Reality:** Image segmentation is crucial for scene understanding in AR applications, helping to identify and track objects in real-time video.

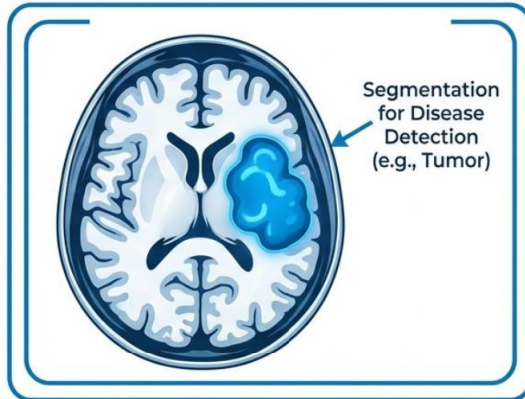


National University of Computer & Emerging Sciences - NUCES - Karachi

Department of Artificial Intelligence

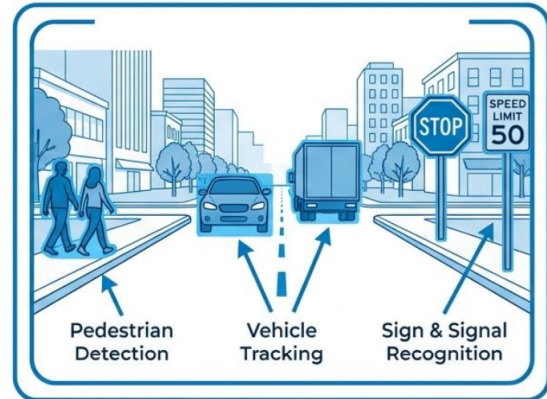
REAL-LIFE EXAMPLES OF IMAGE SEGMENTATION

MEDICAL IMAGING: MRI BRAIN SCAN



Precise Anatomical &
Pathological Segmentation

AUTONOMOUS VEHICLE: OBJECT DETECTION



Scene Parsing & Obstacle
Identification for Navigation

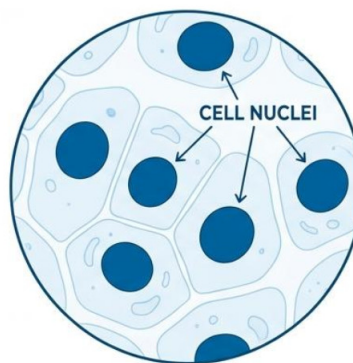
Talha Shahid

SATELLITE IMAGERY



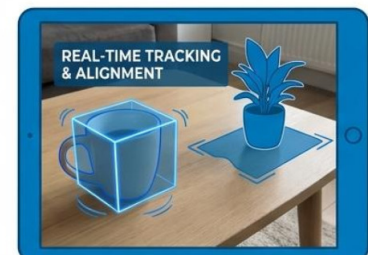
Partitioning landscapes into
distinct segments for land
cover classification.

BIOMEDICAL IMAGING



Accurately isolating specific
cellular structures like nuclei
for analysis.

AUGMENTED REALITY



Precise overlay and interaction
of digital content with
real-world objects.



2.3 Image Segmentation Techniques

Image segmentation is a critical task in computer vision, and various techniques and methodologies are employed to achieve accurate and meaningful segmentations. Here are some common techniques and methodologies used in image segmentation:

2.3.1 Thresholding

Methodology: Divides an image into two regions based on pixel intensity values. Pixels above a certain threshold belong to one region, and those below belong to another.

Use Cases: Simple and effective for segmenting objects with distinct intensity differences.

2.3.1.1 Global Thresholding

Methodology: A single global threshold value is applied to the entire image.

Use Cases: Effective when there is a clear intensity difference between the objects and background.

```
import cv2

image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)
_, binary_mask = cv2.threshold(image, 128, 255, cv2.THRESH_BINARY)
```

2.3.1.2 Adaptive Thresholding

Methodology: Uses different threshold values for different regions of the image, which adapt to local variations in intensity.

Use Cases: Ideal for images with varying lighting conditions.



```
import cv2

image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)
binary_mask = cv2.adaptiveThreshold(image, 255,
cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 11, 2)
```

2.3.1.3 Otsu's Thresholding

Methodology: Automatically selects an optimal threshold value to maximize the inter-class variance between object and background pixels.

Use Cases: Suitable when the distribution of pixel intensities is bimodal.

```
import cv2

image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)
_, binary_mask = cv2.threshold(image, 0, 255, cv2.THRESH_BINARY +
cv2.THRESH_OTSU)
```

2.3.1.4 Color-Based Thresholding

Methodology: Applies thresholding in multiple color channels (e.g., RGB, HSV) to segment based on color.

Use Cases: Useful for segmenting objects with distinct colors.

```
import cv2
import numpy as np

image = cv2.imread('image.jpg')
hsv_image = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
lower_bound = np.array([30, 50, 50])
upper_bound = np.array([60, 255, 255])
mask = cv2.inRange(hsv_image, lower_bound, upper_bound)
```



2.3.1.5 Hysteresis Thresholding (Canny Edge Detector):

Methodology: Utilizes two threshold values, a high threshold to detect strong edges and a low threshold to link weak edges.

Use Cases: Typically used for edge detection but can be adapted for segmentation.

```
import cv2
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)
edges = cv2.Canny(image, 100, 200)
```

2.3.2 Edge Detection

Methodology: Detects boundaries between objects by identifying rapid changes in intensity or color.

Use Cases: Useful for extracting object contours and boundaries.

2.3.3 Region Growing

Methodology: Starts with a seed pixel and expands to neighboring pixels that are similar in some criteria (e.g., intensity, color).

Use Cases: Effective for segmenting regions with uniform characteristics.

```
import cv2
import numpy as np
from matplotlib import pyplot as plt

# Load the image
image = cv2.imread('ents.jpg', cv2.IMREAD_GRAYSCALE)

# Initialize seed point (starting point for region growing)
seed_point = (10, 10)
```



National University of Computer & Emerging Sciences - NUCES - Karachi

Department of Artificial Intelligence

```
# Define a function for region growing
def region_growing(image, seed, threshold):
    # Create an empty mask to store the segmented region
    mask = np.zeros_like(image, dtype=np.uint8)

    # Create a stack for pixel traversal
    stack = [seed]

    # Get the intensity value of the seed point
    seed_intensity = image[seed]

    while stack:
        x, y = stack.pop()

        # Check if the pixel is within the image boundaries
        if x < 0 or x >= image.shape[0] or y < 0 or y >=
image.shape[1]:
            continue

        # Check if the pixel is unvisited
        if mask[x, y] == 0:
            # Check if the intensity difference is below the
threshold
            if abs(int(image[x, y]) - int(seed_intensity)) <
threshold:
                # Add the pixel to the segmented region
                mask[x, y] = 255

                # Add neighboring pixels to the stack
                stack.extend([(x + 1, y), (x - 1, y), (x, y + 1),
(x, y - 1)])

    return mask

# Define a threshold
threshold = 50

# Perform region growing
segmented_image = region_growing(image, seed_point, threshold)

# # Display the segmented image
# cv2.imshow('Segmented Image', segmented_image)
# cv2.waitKey(0)
# cv2.destroyAllWindows()

plt.imshow(segmented_image)
plt.axis('off')
```



```
plt.show()
```

2.3.4 Watershed Segmentation

Methodology: Treats the image as a topographic map and simulates flooding to identify segment boundaries.

Use Cases: Suitable for segmenting touching or overlapping objects.

Steps:

1. Preprocessing

- Convert the input image to grayscale if it's a color image.
- Apply suitable preprocessing techniques like noise reduction and contrast enhancement if necessary.

2. Marker Generation

- Identify markers that represent the regions you want to segment. These markers can be manually created or generated automatically based on specific criteria. Common approaches include:
- Manual placement of markers by the user.
- Thresholding to create markers based on intensity levels.
- Distance transform to create markers around the object centers.

3. Gradient Calculation

- Calculate the gradient of the image to highlight the boundaries of objects. You can use techniques like Sobel or Scharr filters.

4. Marker Labeling

- Label the markers created in step 2 using different integer values.

5. Watershed Transformation

- Treat the gradient image as a topographic surface where high gradient values represent peaks.
- Fill basins in the topographic surface with different colors, starting from the labeled markers.
- The boundaries where the basins meet represent the segmented regions.

6. Post-processing

- Optionally, you can apply post-processing techniques like morphological operations (erosion, dilation) to refine the segmentation.



National University of Computer & Emerging Sciences - NUCES - Karachi

Department of Artificial Intelligence

```
import cv2
import numpy as np
from matplotlib import pyplot as plt

# Load the image
image = cv2.imread('dog.jpeg')
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Threshold the image to create markers (you can adjust the
threshold value)
_, thresh = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV +
cv2.THRESH_OTSU)

# Perform morphological operations to remove noise
kernel = np.ones((5, 5), np.uint8)
opening = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel,
iterations=2)

# Create sure background and sure foreground markers
sure_bg = cv2.dilate(opening, kernel, iterations=3)
dist_transform = cv2.distanceTransform(opening, cv2.DIST_L2, 5)
_, sure_fg = cv2.threshold(dist_transform, 0.2 *
dist_transform.max(), 255, 0)

# Subtract sure foreground from sure background to get unknown
region
sure_fg = np.uint8(sure_fg)
unknown = cv2.subtract(sure_bg, sure_fg)

# Label markers for watershed algorithm
_, markers = cv2.connectedComponents(sure_fg)
markers = markers + 1
markers[unknown == 255] = 0

# Apply watershed algorithm
cv2.watershed(image, markers)
image[markers == -1] = [255, 0, 0] # Mark boundaries with red
color

plt.imshow(image)
plt.axis('on')
plt.show()
```



2.3.5 Clustering-Based Segmentation

Methodology: Groups pixels into clusters based on feature similarity using techniques like K-Means, Mean-Shift, or DBSCAN.

Use Cases: Effective for segmenting objects with complex characteristics.

Steps:

1. Feature Extraction

- a. Choose the feature(s) that represent each pixel's characteristics. For color images, these features could be RGB values or color spaces like LAB or HSV. For grayscale images, intensity values are commonly used.

2. Cluster Initialization

- a. Decide on the number of clusters (K) you want to create. This choice depends on the specific image and application.
- b. Initialize K cluster centroids. These centroids represent the initial cluster centers, which can be randomly chosen or based on some heuristic.

3. Clustering Algorithm (K-Means)

- a. Iterate through the following steps until convergence.
- b. Assignment Step: Assign each pixel to the nearest cluster centroid based on feature similarity (e.g., Euclidean distance).
- c. Update Step: Recalculate the cluster centroids as the mean of the pixels in each cluster.
- d. Convergence is reached when the cluster assignments no longer change significantly.

4. Segmentation

- a. After convergence, the image is segmented based on the final cluster assignments. Each cluster represents a segment or region in the image.

5. Post-processing

- a. Optionally, apply post-processing techniques to refine the segmentation, such as merging or splitting clusters based on specific criteria.



```
import cv2
import numpy as np
from matplotlib import pyplot as plt

# Load the image
image = cv2.imread('dogimg.jpg')

# Reshape the image to a 2D array of pixels
pixel_values = image.reshape((-1, 3))

# Convert to float32 for K-Means clustering
pixel_values = np.float32(pixel_values)

# Define criteria and apply K-Means clustering
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 100,
0.2)
K = 4 # Number of clusters (you can adjust this)
_, labels, centers = cv2.kmeans(pixel_values, K, None, criteria, 10,
cv2.KMEANS_RANDOM_CENTERS)

# Convert back to 8-bit values
centers = np.uint8(centers)
segmented_image = centers[labels.flatten()]

# Reshape the segmented image back to its original shape
segmented_image = segmented_image.reshape(image.shape)

# Display the segmented image
cv2.imshow('Segmented Image', segmented_image)
cv2.waitKey(0)
cv2.destroyAllWindows()

plt.imshow(segmented_image)
plt.axis('on')
plt.show()
```

2.3.6 Deep Learning-Based Segmentation

Methodology: Utilizes deep learning models, such as Convolutional Neural Networks (CNNs), for semantic and instance segmentation.

Use Cases: Suitable for complex tasks and large datasets.



National University of Computer & Emerging Sciences - NUCES - Karachi

Department of Artificial Intelligence

Frameworks: TensorFlow, PyTorch, and libraries like Keras provide pre-trained models and tools for deep learning-based segmentation.

2.3.7 Active Contour Models (Snakes)

Methodology: Deformable models that evolve and adapt to object boundaries based on image gradients and constraints.

Use Cases: Effective for object tracking and boundary refinement.

Talha Shahid